Trabajo

Comparar la implementación de un diccionario de datos físico en diferentes SGBD

Estudiantes

Mariana López Díaz.

Sara Ximena Machado.

Asignatura

Bases de Datos.

Docente

Oralia Cortes Grajales.

Institucion Universitaria Pascual Bravo

Facultad de Ingeniería

Tecnología de desarrollo de Software

Medellín

2026

Este informe tiene como finalidad comparar la adaptación del diccionario de datos físico del proyecto Pascualina en tres sistemas gestores de bases de datos (SGBD): PostgreSQL, Microsoft SQL Server y MySQL.

Pascualina es una propuesta de red social académica orientada a la comunidad estudiantil. Su estructura permite gestionar información de estudiantes, publicaciones, comentarios, intereses, habilidades, grupos, eventos, membresías, seguimientos y participaciones en actividades.

El proyecto cuenta con un modelo de base de datos compuesto por doce tablas relacionadas mediante claves primarias, claves foráneas y restricciones de integridad. Para desarrollar esta actividad, se elaboró un diccionario de datos general y se adaptó su estructura física a las características de cada SGBD, teniendo en cuenta los tipos de datos, la generación de identificadores, las relaciones y las restricciones.

Adicionalmente, se incorporó el campo `metadatos` en la tabla `evento`, con el propósito de estudiar el manejo de información semiestructurada mediante JSON o mecanismos equivalentes. Esta incorporación permite analizar posibles usos futuros relacionados con Big Data e Internet de las Cosas (IoT).

Finalmente, se presenta una comparación de los tres sistemas, las principales dificultades identificadas durante la adaptación, sus ventajas y desventajas, y las conclusiones individuales y grupales del proceso.

Objetivo general

Comparar la implementación del diccionario de datos físico del proyecto Pascualina en PostgreSQL, Microsoft SQL Server y MySQL, identificando sus características, diferencias técnicas, ventajas, desventajas y posibilidades de mejora.

Objetivos específicos

- Seleccionar y justificar tres sistemas gestores de bases de datos para representar la estructura física del proyecto Pascualina.
- Elaborar un diccionario de datos general que describa las tablas, campos, tipos de datos y relaciones de la base de datos.
- Adaptar el diccionario físico y los scripts DDL a la sintaxis y características de cada SGBD.
- Comparar el manejo de claves, restricciones, fechas, textos e información semiestructurada.
- Identificar dificultades técnicas y proponer mejoras para la implementación y documentación del proyecto.

Justificación de la elección de los tres SGBD

Para el desarrollo de esta actividad se seleccionaron PostgreSQL, Microsoft SQL Server y MySQL porque permiten implementar bases de datos relacionales y ofrecen diferentes alternativas para administrar la información del proyecto Pascualina.

PostgreSQL

PostgreSQL fue seleccionado porque es el sistema utilizado previamente para implementar la base de datos Pascualina. Esto permite aprovechar el modelo físico existente y utilizarlo como referencia para adaptar la estructura a los otros dos sistemas.

Entre sus características se destacan el uso de claves primarias y foráneas, restricciones de integridad, columnas de identidad y tipos de datos como `TEXT`, `TIMESTAMP` y `JSONB`.

Su tipo `JSONB` permite almacenar información semiestructurada en formato JSON y facilita la consulta de sus elementos. Además, PostgreSQL es de código abierto y cuenta con herramientas para administrar, consultar y respaldar bases de datos.

Justificación: se eligió como sistema de referencia porque permite conservar la continuidad del proyecto y comparar las adaptaciones realizadas en los otros SGBD.

Microsoft SQL Server

Microsoft SQL Server fue seleccionado como alternativa para evaluar cómo se representa la misma estructura relacional utilizando herramientas y tipos de datos propios de este sistema.

En el diseño propuesto se utilizaron elementos como `INT IDENTITY`, `DATETIME2`, `NVARCHAR(MAX)` y restricciones `CHECK`. Para almacenar metadatos JSON se planteó utilizar `NVARCHAR(MAX)` junto con la función `ISJSON`, que permite validar el formato del contenido en las versiones compatibles.

SQL Server ofrece herramientas de administración, mecanismos de seguridad y opciones de integración con otras tecnologías del entorno Microsoft. Sin embargo, algunas de sus herramientas y funciones avanzadas pueden depender de la edición y de la licencia utilizada.

Justificación: se eligió para comparar una implementación basada en tecnologías de Microsoft con la estructura previamente diseñada en PostgreSQL.

MySQL

MySQL fue seleccionado porque es un sistema gestor relacional ampliamente utilizado en aplicaciones web y permite trabajar con tablas, relaciones, restricciones y tipos de datos equivalentes a los empleados en Pascualina.

En el script propuesto se utilizaron elementos como `AUTO_INCREMENT`, `DATETIME`, `TEXT`, `ENGINE=InnoDB` y el tipo nativo `JSON`.

InnoDB permite establecer claves foráneas y aplicar acciones referenciales, mientras que el tipo JSON facilita el almacenamiento de información semiestructurada.

Justificación: se eligió para comparar una alternativa relacional utilizada en numerosos proyectos web y observar cómo se adapta la misma estructura física a su sintaxis.

Resumen de la selección

Criterio	PostgreSQL	SQL Server	MySQL
Identificadores automáticos	`GENERATED AS IDENTITY`	`IDENTITY(1,1)`	`AUTO_INCREMENT`
Información semiestructurada	`JSONB`	`NVARCHAR(MAX)` con validación JSON	`JSON`
Fecha y hora	`TIMESTAMP` y `TIME`	`DATETIME2` y `TIME`	`DATETIME` y `TIME`
Claves foráneas	Sí	Sí	Sí, con InnoDB
Licenciamiento	Código abierto	Depende de la edición	Código abierto en su edición comunitaria, con otras opciones comerciales

La comparación demuestra que los tres sistemas pueden representar la estructura relacional de Pascualina, aunque es necesario adaptar la sintaxis, los tipos de datos y determinadas restricciones a cada plataforma.

Explicación paso a paso de la implementación.

Análisis del modelo de datos existente

El primer paso consistió en revisar la estructura de la base de datos Pascualina desarrollada anteriormente en PostgreSQL. Se identificaron las doce tablas, sus atributos, las claves primarias, las claves foráneas y las restricciones de integridad.

Las tablas identificadas son:

1. `estudiante`
2. `publicacion`
3. `comentario`

4. `interes`
5. `estudiante_interes`
6. `habilidad`
7. `estudiante_habilidad`
8. `seguimiento`
9. `grupo`
10. `membresia`
11. `evento`
12. `participacion_evento`

A partir de esta revisión, se estableció una estructura de referencia que debía conservarse en los tres sistemas gestores.

Elaboración del diccionario de datos general

Se elaboró un diccionario general para documentar el propósito de las tablas, los atributos que las componen y las relaciones principales entre ellas.

Este documento funciona como referencia común para las tres implementaciones. Su finalidad es evitar que cada SGBD represente un modelo diferente y garantizar que las adaptaciones mantengan la misma estructura lógica.

Posteriormente, se elaboraron tres diccionarios físicos independientes, en los cuales se documentaron los tipos de datos, las claves y las restricciones correspondientes a cada sistema.

Implementación del diccionario físico en PostgreSQL

PostgreSQL se utilizó como punto de referencia debido a que la base de datos original del proyecto fue desarrollada en este sistema.

Los pasos de adaptación fueron los siguientes:

1. Se revisaron las doce tablas existentes y se identificaron los tipos de datos y las restricciones utilizadas.
2. Se documentaron los campos y sus características, incluyendo longitud, obligatoriedad y valores predeterminados cuando correspondía.
3. Se representaron los identificadores automáticos mediante columnas `GENERATED AS IDENTITY`, utilizando `GENERATED BY DEFAULT` en los identificadores de `seguimiento` y `membresia` para mantener la configuración definida en el diseño.
4. Se conservaron las claves primarias simples y compuestas según correspondía a cada tabla.
5. Se documentaron las claves foráneas para representar las relaciones entre estudiantes, publicaciones, comentarios, grupos, intereses, habilidades y eventos.

6. Se definieron las acciones referenciales `ON DELETE CASCADE` en las relaciones que requieren eliminar los registros dependientes cuando se elimina el registro principal.
7. Se incorporó el campo `metadatos JSONB` en la tabla `evento` dentro del nuevo diseño propuesto.
8. Se preparó el script `pascualina-postgresql.sql`, que contiene las instrucciones `CREATE TABLE` necesarias para construir la estructura documentada.

El uso de `JSONB` permite almacenar propiedades adicionales de un evento, por ejemplo, información sobre el dispositivo que registra una actividad, características de un taller o datos complementarios que no requieren columnas independientes.

El script se organizó creando primero las tablas principales y posteriormente las tablas dependientes, para facilitar la definición de las claves foráneas.

La preparación de este script no implica modificar la base de datos original, ya que se diseñó como un archivo independiente para la nueva actividad.

Implementación del diccionario físico en Microsoft SQL Server

Para adaptar el modelo a Microsoft SQL Server se conservaron las doce tablas y sus relaciones, pero se modificaron los tipos de datos y la sintaxis de generación de identificadores.

Los pasos fueron los siguientes:

1. Se tomó el diccionario general como referencia para conservar los nombres de las tablas, los atributos y sus relaciones.
2. Se adaptaron los identificadores automáticos utilizando `INT IDENTITY(1,1)`.
3. Se representaron los campos de texto extensos mediante `NVARCHAR(MAX)` cuando se requería almacenar contenido amplio.
4. Se utilizaron `DATETIME2`, `DATE` y `TIME` para representar los atributos temporales.
5. Se declararon las claves primarias, las claves foráneas, las restricciones `UNIQUE` y las restricciones `CHECK`.
6. Se incorporó el campo `metadatos NVARCHAR(MAX)` en la tabla `evento`.
7. Se propuso una restricción `CHECK` con `ISJSON(metadatos)` para validar que el contenido almacenado tenga formato JSON en las versiones compatibles.
8. Se organizó el archivo `pascualina-sql-server.sql` con las instrucciones de creación de las doce tablas.

La principal adaptación corresponde al almacenamiento de información semiestructurada. A diferencia de PostgreSQL y MySQL, el diseño presentado utiliza una columna de texto para guardar el JSON y una función de validación para comprobar su formato.

Esta alternativa permite representar metadatos sin crear una columna independiente por cada propiedad adicional. No obstante, el procesamiento y la consulta de estos datos requieren utilizar las funciones JSON disponibles en SQL Server.

El script fue preparado para ejecutarse en una base de datos de pruebas compatible con la sintaxis de SQL Server, sin utilizarlo sobre la base original de PostgreSQL.

Implementación del diccionario físico en MySQL

Para MySQL se adaptó el modelo de Pascualina conservando las doce tablas y sus relaciones principales.

El procedimiento fue el siguiente:

1. Se revisó el diccionario general y se identificaron los campos que debían mantenerse en la estructura física.
2. Se adaptaron los identificadores automáticos mediante `INT AUTO_INCREMENT`.
3. Se utilizaron tipos como `VARCHAR`, `TEXT`, `DATE`, `TIME` y `DATETIME` según las características de cada atributo.
4. Se declararon las claves primarias y las restricciones de unicidad.
5. Se definieron las claves foráneas para conservar las relaciones entre las tablas.
6. Se seleccionó el motor `InnoDB` para las tablas, debido a su soporte de claves foráneas y acciones referenciales.
7. Se incorporó el campo `metadatos JSON` en la tabla `evento`.
8. Se preparó el archivo `pascualina-mysql.sql`, con las instrucciones DDL correspondientes a las doce tablas.

El tipo `JSON` de MySQL permite almacenar documentos JSON y validar su sintaxis. Esto facilita la incorporación de información adicional de los eventos sin tener que modificar la estructura de la tabla cada vez que se agrega una nueva propiedad semiestructurada.

Finalmente, se organizó el script para crear primero las tablas principales y después las tablas que dependen de ellas.

Organización de los archivos del proyecto

Los documentos y scripts se organizaron en GitHub dentro de la carpeta `tarea4`, con el propósito de separar los diccionarios, las implementaciones y la documentación.

La estructura desarrollada es la siguiente:

- `tarea4/diccionarios/`: contiene el diccionario general y los tres diccionarios físicos.
- `tarea4/ddl/scripts-ddl-sgbd-1/`: contiene el script DDL de PostgreSQL.
- `tarea4/ddl/scripts-ddl-sgbd-2/`: contiene el script DDL de SQL Server.
- `tarea4/ddl/scripts-ddl-sgbd-3/`: contiene el script DDL de MySQL.
- `tarea4/informe/`: está destinado al informe técnico comparativo.
- `tarea4/video/`: está destinado a la evidencia audiovisual de la sustentación.

Esta organización facilita la consulta de los archivos y permite distinguir la documentación general de las implementaciones específicas de cada SGBD.

Dificultades encontradas y análisis técnico

Durante la adaptación del diseño se identificaron varios aspectos que requieren especial atención al trabajar con sistemas gestores diferentes.

Diferencias en los tipos de datos

Una de las principales dificultades consiste en que cada SGBD utiliza tipos de datos y convenciones propias. Por ejemplo, PostgreSQL utiliza `GENERATED AS IDENTITY`, SQL Server utiliza `IDENTITY(1,1)` y MySQL utiliza `AUTO_INCREMENT` para generar identificadores automáticamente.

También existen diferencias en la representación de fechas, horas y textos extensos. Por esta razón, fue necesario adaptar los tipos de datos del diccionario físico sin cambiar el propósito de los atributos.

Manejo de información semiestructurada

La incorporación de `metadatos` en la tabla `evento` permitió identificar diferencias importantes entre los tres sistemas.

- PostgreSQL permite emplear `JSONB` para almacenar y consultar información JSON.
- SQL Server puede representar JSON en columnas de texto, con validación mediante `ISJSON` en versiones compatibles.
- MySQL cuenta con el tipo nativo `JSON`.

Esta diferencia exige documentar cómo se almacena y valida la información en cada plataforma. Además, las funciones de consulta y los mecanismos de indexación no son idénticos.

Conservación de las relaciones

Las claves foráneas deben declararse cuando las tablas referenciadas ya existen. Por ello, los scripts se organizaron creando primero las tablas principales y después las tablas dependientes.

También fue necesario conservar las restricciones de unicidad y las claves primarias compuestas de las tablas que relacionan estudiantes con intereses, habilidades y eventos.

Las acciones `ON DELETE CASCADE` deben utilizarse de acuerdo con las reglas del modelo, porque pueden eliminar registros dependientes cuando se elimina un registro principal. Las relaciones que no tienen esta acción conservan el comportamiento predeterminado del sistema gestor.

Diferencias en la validación de datos

Las restricciones y funciones disponibles varían entre sistemas y versiones. Por ello, la validación del campo JSON y de las restricciones `CHECK` debe verificarse en el SGBD y la versión utilizados.

Además, un script válido en PostgreSQL no puede ejecutarse directamente en SQL Server o MySQL sin adaptar su sintaxis. Esto demuestra la importancia de mantener un archivo DDL independiente para cada plataforma.

Las dificultades descritas corresponden a los aspectos técnicos identificados durante el diseño y la adaptación de los archivos. Para determinar los errores reales de ejecución y confirmar la compatibilidad completa, es necesario ejecutar y probar cada script en su respectivo SGBD.

Ventajas y desventajas de cada SGBD

PostgreSQL

Ventajas

- Es el sistema utilizado como referencia en el proyecto original.
- Ofrece soporte para claves primarias, claves foráneas y restricciones de integridad.
- Cuenta con el tipo `JSONB` para manejar información semiestructurada.
- Es de código abierto y dispone de herramientas de administración y respaldo.

Desventajas

- La sintaxis de sus identificadores automáticos y tipos de datos no es directamente compatible con los otros sistemas.
- La migración hacia otra plataforma requiere adaptar los scripts.
- El manejo de JSON y las funciones específicas de PostgreSQL pueden requerir ajustes al migrar la aplicación.

Microsoft SQL Server

Ventajas

- Ofrece herramientas de administración y mecanismos de seguridad para entornos empresariales.
- Permite definir restricciones y relaciones entre tablas.
- Dispone de funciones para consultar y validar información JSON.
- Se integra con herramientas del ecosistema Microsoft.

Desventajas

- Algunas funciones y opciones dependen de la edición, la versión y el licenciamiento.

- El manejo de JSON mediante columnas de texto requiere definir mecanismos de validación y consulta.
- Su sintaxis difiere de PostgreSQL y MySQL, por lo que es necesario adaptar los scripts.

MySQL

Ventajas

- Es ampliamente utilizado en aplicaciones web.
- Cuenta con el motor InnoDB para gestionar relaciones y restricciones referenciales.
- Dispone del tipo de dato `JSON`.
- Permite utilizar `AUTO_INCREMENT` para generar identificadores automáticamente.

Desventajas

- La sintaxis y algunas restricciones deben ajustarse a la versión de MySQL utilizada.
- Las funciones de JSON y sus posibilidades de indexación difieren de las de PostgreSQL y SQL Server.
- La migración de scripts desde otros SGBD requiere revisar los tipos de datos, las restricciones y las acciones referenciales.

Comparación general

Aspecto	PostgreSQL	SQL Server	MySQL
Adaptación del modelo relacional	Compatible mediante DDL propio	Compatible mediante DDL propio	Compatible mediante DDL propio
Generación de identificadores	`IDENTITY`	`IDENTITY`	`AUTO_INCREMENT`
Manejo de JSON en el diseño	`JSONB`	Texto con funciones de JSON	Tipo nativo `JSON`
Claves foráneas	Sí	Sí	Sí, con InnoDB
Principal consideración	Adaptar sintaxis al migrar	Validar funciones según versión	Revisar versión, motor y restricciones

La comparación evidencia que los tres SGBD permiten representar la estructura de Pascualina, pero no son completamente intercambiables. La selección final debe considerar las necesidades del proyecto, las herramientas disponibles, la experiencia del equipo, el rendimiento requerido y el mantenimiento futuro.

Relación del campo JSON con Big Data en IoT

En la tabla `evento` se propuso incorporar el atributo `metadatos`, destinado a almacenar información complementaria relacionada con las actividades académicas y sociales de Pascualina.

Por ejemplo, un evento podría incluir datos adicionales como el dispositivo utilizado para registrar la asistencia, el origen de una notificación, información técnica de una actividad o características complementarias del evento.

Estos datos pueden variar entre eventos. En lugar de crear una columna independiente para cada propiedad posible, el uso de JSON permite agruparlas en un documento semiestructurado.

La propuesta se relaciona con Big Data porque facilita representar datos con una estructura variable, que pueden generarse en grandes cantidades cuando la aplicación crece. También tiene relación con IoT, ya que en una futura ampliación podrían recibirse metadatos procedentes de dispositivos conectados, como sensores de asistencia o equipos que registren información de una actividad.

Sin embargo, incorporar una columna JSON no convierte por sí sola la base de datos en una plataforma Big Data o IoT. Para ello serían necesarios mecanismos adicionales de captura, procesamiento, almacenamiento y análisis de datos.

Conclusión individual de la integrante Sara Machado

La elaboración de este proyecto me permitió comprender la importancia de adaptar el diccionario de datos físico a las características del sistema gestor utilizado. Aprendí que, aunque una base de datos puede conservar las mismas tablas y relaciones, cada SGBD tiene su propia sintaxis y ofrece diferentes alternativas para representar los tipos de datos y las restricciones.

También comprendí la importancia de documentar las claves primarias y foráneas, porque permiten mantener la integridad de la información. La comparación entre PostgreSQL, SQL Server y MySQL fortaleció mis conocimientos sobre diseño físico de bases de datos y organización de scripts DDL.

Como propuesta de mejora, considero importante ejecutar y validar cada script en su respectivo sistema gestor, documentar los errores que se presenten y realizar pruebas de integridad antes de considerar terminada la implementación.

Conclusión individual de la integrante Mariana López

El desarrollo de la actividad me permitió reconocer que el diccionario de datos es una herramienta fundamental para documentar y comprender la estructura de una base de datos. La adaptación del proyecto Pascualina a tres SGBD me ayudó a identificar las diferencias entre los tipos de datos, los identificadores automáticos y los mecanismos para gestionar información semiestructurada.

Además, comprendí que el uso de JSON puede ser útil cuando se necesita almacenar información adicional que cambia entre registros, como los metadatos de los eventos. Sin embargo, su incorporación debe responder a una necesidad real y acompañarse de mecanismos adecuados de validación y consulta.

Como mejora, propongo ampliar las pruebas de funcionamiento, comparar las consultas y el rendimiento de cada sistema con datos de prueba equivalentes y mantener actualizada la documentación técnica del repositorio.

Conclusión general

Como equipo, concluimos que la adaptación del diccionario de datos físico de Pascualina a PostgreSQL, Microsoft SQL Server y MySQL permite comprender las semejanzas y diferencias entre tres sistemas gestores de bases de datos relacionales.

El proceso permitió organizar la estructura de doce tablas, documentar sus atributos y relaciones, preparar scripts DDL independientes y reconocer la importancia de adaptar los tipos de datos y las restricciones a cada plataforma. También permitió estudiar diferentes mecanismos para almacenar información semiestructurada mediante el atributo `metadatos` de la tabla `evento`.

La principal enseñanza es que el diseño lógico constituye una referencia común, pero la implementación física requiere decisiones específicas para cada SGBD. Por tanto, la documentación, la organización de los archivos y la verificación de las restricciones son elementos esenciales para mantener la coherencia del proyecto.

Como propuesta de mejora grupal, se plantea ejecutar los scripts en entornos de prueba independientes, verificar la creación de las doce tablas, comprobar las claves primarias y foráneas, probar el almacenamiento de datos JSON y documentar los resultados. También se propone utilizar datos de prueba equivalentes para comparar consultas, rendimiento y facilidad de mantenimiento.

De esta manera, Pascualina podrá contar con una documentación técnica más completa y una base para futuras ampliaciones, sin comprometer la base de datos original.

Referencias y recursos del proyecto

- Proyecto Pascualina. Diccionario de datos general y diccionarios físicos elaborados para la tarea 4.
- Proyecto Pascualina. Scripts DDL de PostgreSQL, Microsoft SQL Server y MySQL.
- Documentación oficial de PostgreSQL: https://www.postgresql.org/docs/
- Documentación oficial de Microsoft SQL Server: https://learn.microsoft.com/es-es/sql/sql-server/
- Documentación oficial de MySQL: https://dev.mysql.com/doc/